

Final Term Study Plan

Seven days to the term exam. Built from the official course outline: what is actually examinable, what it is worth, and what to do on each of the seven days.

READ THIS FIRST

The term exam is the whole game. It is 60% of the final term, and the final term is 60% of the course — so this one paper decides **36% of your entire course grade**. Nothing else you can still influence comes close.

The final term outweighs the midterm. The course is 40% midterm and 60% final term. A poor midterm does not decide your grade; this term does more of the deciding than the one you already sat.

Four topics are new to you. Your revision sheet covers PHP basics, forms, sessions, cookies, AJAX and JSON. It does *not* cover PHP & MySQL, MVC, web security, or API fundamentals — and all four are on the final. They are Days 1 to 3 below.

1 What the marks actually look like

From the course outline, section XV. Worth knowing before you decide where the hours go.

Final-term component	Weight in final term	Share of course grade	Can you still change it?
Term Exam	60%	36%	Yes — entirely. This is the plan.
2 Quizzes (best 1 counts)	20%	12%	Already sat
Project Prototype	10%	6%	Yes — DoctorConnect is built; make sure it runs on demo day
Attendance & Performance	10%	6%	Fixed

Two pieces of good news. Only the *better* of your two quizzes counts, so a weak one is already discarded. And the project prototype is 10% that is effectively banked — DoctorConnect exists, runs, and passes its tests. Protect that by checking it still starts, and spend everything else on the exam.

2 What is examinable

Weeks 8 to 13 of the course outline — everything taught after the midterm.

Week	Topic	In your revision sheet?	Where it is covered
8	Introduction to PHP	Yes	Revision sheet, blocks 1–3
9	PHP Form Submission, Form Validation	Yes	Revision sheet, block 3
10	PHP Session and Cookie	Yes	Revision sheet, blocks 1–2
11	PHP & MySQL, MVC	No	Day 1 and Day 3 below
12	Data Transportation: AJAX and JSON	Yes	Revision sheet, block 7
13	Web Security, API Fundamentals	No	Day 2 and Day 3 below

The gap that matters: two of the six examinable weeks are topics you have never revised. That is roughly a third of the paper. Days 1–3 exist to close it, which is why the new material comes before re-reading anything you have already seen.

3 The seven days

Counting down to the exam. Two to three hours a day; the load is heaviest early on purpose, so the last two days are revision rather than learning.

Day 1 — PHP & MySQL

the biggest new topic · ~2.5 h

- Learn the connection, and all four CRUD operations (section 4 below).
- Learn prepared statements properly — they answer both a MySQL question *and* a security question.
- Open `doctorconnect/db.php` and `manage_doctors.php`. Every operation you just learned is running in there.
- Write a connect + SELECT + display loop from memory before you stop.

Day 2 — Web Security

new · heavily examinable as theory · ~2 h

- The five attacks and their fixes (section 5 below). Mostly written answers, so this is cheaper than it looks.
- Learn `password_hash()` and `password_verify()` — near-certain question.
- Your project already defends against all of these: prepared statements in 16 files, hashing, `htmlspecialchars`, `session_regenerate_id`. Open `auth.php` and name each defence out loud.

Day 3 — MVC and APIs

new, but both are small · ~2 h

- MVC: three definitions and why the separation helps (section 6). Pure theory — your project is *not* MVC, so there is nothing to look at.
- API fundamentals: REST, the four HTTP methods, the status codes (section 7).
- `workshop-registration/get_workshops.php` is effectively a small API endpoint. Read it with the new vocabulary in mind.

Day 4 — Sessions, cookies, forms

re-drill · ~2 h

- Blocks 1–4 of the revision sheet: cookies, sessions, form validation, file upload.
- Write each from memory. You revised these for the quiz, so this is maintenance, not learning.
- Then the three comparison tables — server vs client, GET vs POST, session vs cookie.

Day 5 — Practice, not reading

~2.5 h

- All the output tracing rules, closed book.
- Write four full answers by hand: a form handler, a login with session guard, a file upload, a MySQL SELECT and display.
- Mark yourself honestly. Whatever you could not write becomes tomorrow's list.

Day 6 — Close the gaps

~2 h

- Only the topics you failed on Day 5. Do not re-read what you already got right.
- Re-test those specific ones at the end of the session.
- Check DoctorConnect still starts — XAMPP Apache and MySQL, then load `login.php`. That protects 6% of your grade for five minutes of effort.

- The three comparison tables and the security list. Pure recall, no new material.
- Skim the code blocks once. Do not start anything you have not already learned.
- Sleep. A tired pass over familiar material scores worse than a rested one.

4 Day 1 material — PHP & MySQL

Week 11. The largest new topic and the most likely long-form code question.

Connect, then always check

```
<?php
$conn = mysqli_connect("localhost", "root", "", "school_db");

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
?>
```

SELECT and display — the most common question

```
$result = mysqli_query($conn, "SELECT id, name, email FROM students");

while ($row = mysqli_fetch_assoc($result)) {
    echo $row["name"] . " - " . $row["email"] . "<br>";
}

echo mysqli_num_rows($result);    // how many rows came back
```

INSERT, UPDATE, DELETE

```
mysqli_query($conn, "INSERT INTO students (name, email)
                    VALUES ('Minhaz', 'm@mail.com')");

mysqli_query($conn, "UPDATE students SET name = 'Rahim' WHERE id = 1");

mysqli_query($conn, "DELETE FROM students WHERE id = 1");

mysqli_close($conn);
```

Prepared statements — the safe way, and worth double marks

```
$stmt = mysqli_prepare($conn, "INSERT INTO students (name, email) VALUES (?, ?)");

mysqli_stmt_bind_param($stmt, "ss", $name, $email);    // "ss" = two strings
mysqli_stmt_execute($stmt);

// Reading results back from a prepared statement
$stmt = mysqli_prepare($conn, "SELECT * FROM students WHERE email = ?");
mysqli_stmt_bind_param($stmt, "s", $email);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$row    = mysqli_fetch_assoc($result);
```

The bind_param type letters: `s` string · `i` integer · `d` double · `b` blob. One letter per `?`, in order. Two placeholders and one letter is an error.

Why it earns marks twice: a prepared statement sends the SQL and the data separately, so user input can never be read as SQL. That is the textbook answer to “how do you prevent SQL injection” as well as a MySQL question.

5 Day 2 material — Web Security

Week 13. Mostly written answers, which makes it cheap marks for the time spent.

Attack	What it is	The fix
SQL Injection	User input is treated as SQL, e.g. typing <code>' OR '1'='1</code> into a login box	Prepared statements with <code>bind_param</code>
XSS (Cross-Site Scripting)	User input containing <code><script></code> is echoed back and runs in another user's browser	<code>htmlspecialchars()</code> on all output
CSRF (Cross-Site Request Forgery)	A user who is logged in is tricked into submitting a request they did not intend	A random token in every form, checked on submit
Session hijacking	An attacker steals or fixes the session ID and becomes that user	<code>session_regenerate_id(true)</code> on login; HttpOnly cookies; HTTPS
Weak password storage	Passwords kept as plain text or MD5, readable if the database leaks	<code>password_hash()</code> and <code>password_verify()</code>

Password handling — near-certain question

```
// Registering
$hash = password_hash($password, PASSWORD_DEFAULT);

// Logging in
if (password_verify($input, $user["password"])) {
    session_regenerate_id(true); // stops session fixation
    $_SESSION["user_id"] = $user["id"];
}
```

Say this if asked why hashing: hashing is *one-way*. You can never turn a hash back into the password — you can only hash the attempt and compare. So even if the database leaks, the passwords are not exposed. Never use MD5 or SHA1 for passwords; they are too fast to compute and are considered broken for this purpose.

6 Day 3 material — MVC

Week 11. Theory only — DoctorConnect is procedural, so there is no example of this in your own work.

Layer	Responsibility	In a PHP app
Model	The data and the rules around it. Talks to the database. Knows nothing about how it is displayed.	A class or file holding the SQL queries
View	What the user sees. Displays data it is handed. Contains no business logic.	The HTML template
Controller	Receives the request, asks the Model for data, chooses which View to show.	The script the form posts to

Why MVC is used: separation of concerns, so a change to the layout cannot break the database code; easier testing, because the Model can be tested without a browser; and easier teamwork, since one person can work on views while another works on data.

If asked to compare: in a procedural PHP page the SQL, the logic and the HTML all sit in one file — quick to write, hard to maintain. MVC splits them into three, which costs more setup but scales to larger applications and larger teams.

7 Day 3 material — API fundamentals

Week 13. Small topic, mostly definitions and two lists.

API = Application Programming Interface. A defined way for one program to request data or actions from another, without knowing how it works inside. **REST** is a style of API where each resource has its own URL and you act on it with standard HTTP methods.

Method	Does	Example
GET	Read data	GET /students — list all students
POST	Create something new	POST /students — add a student
PUT	Update an existing item	PUT /students/5
DELETE	Remove an item	DELETE /students/5

Status codes worth memorising

Code	Meaning	Code	Meaning
200	OK — request succeeded	401	Unauthorized — not logged in
201	Created	404	Not Found
400	Bad Request	500	Internal Server Error

APIs almost always exchange **JSON**, which is why AJAX, JSON and APIs are taught together — `json_encode()` on the server, `fetch()` and `res.json()` on the client.

Before the exam — tick these off

Each is something you write on blank paper. If you cannot, that is where the next hour goes.

- ☐ Connect to MySQL, SELECT rows, and display them in a loop.
- ☐ An INSERT using a prepared statement with `bind_param()`.
- ☐ The five attacks and their fixes, from memory.
- ☐ `password_hash` and `password_verify`, and why hashing is one-way.
- ☐ Model, View, Controller — one sentence each, plus why the split helps.
- ☐ The four HTTP methods and six status codes.
- ☐ Cookie set, read and delete.
- ☐ A login page and a session-guarded page.
- ☐ A form handler with empty-field and email validation.

- ☐ A file upload with `enctype` and `move_uploaded_file`.
- ☐ The three comparison tables.
- ☐ The tracing rules: `"5" + 5` vs `"5" . 5`, `$x++` vs `++$x`, `do...while`.
- ☐ DoctorConnect still starts and loads `login.php`.

In the exam hall: the paper is 60% of the final term, so treat every question as worth attempting. If one stalls you, write the structure anyway — the `<?php`, the `if (isset(...))`, the braces, the variable names. Partial marks are real; a blank answer scores nothing.